

REPORT 4

Documento de Projeto

IDENTIFICAÇÃO

Nº	NOME	e-mail	Telefone
212031	Felipe Rusig de Paiva	212031@facens.br	+55 15 98111-6699
223618	Guilherme Massayuki Yokoda de Moraes	223618@facens.br	+55 15 99707-8253
222410	Leonardo Rossi de Oliveira	222410@facens.br	+55 15 99625-1414

TÍTULO: Benchmarking de persistência poliglota em arquiteturas monolíticas e de microserviços para prontuários eletrônicos emergenciais.

LÍDER DO GRUPO: Leonardo Rossi de Oliveira

ORIENTADOR: Prof. Marco Antonio Montebello Junior

Data da Entrega: 18/05/2026

Visto do Orientador

SUMÁRIO

1	INTRODUÇÃO.....	2
2	TRABALHOS CORRELATOS	5
3	O QUE SERÁ FEITO	7
4	O QUE NÃO SERÁ FEITO	9
5	BENEFÍCIOS	10
6	METAS PARA O TCC 2	11
7	RECURSOS UTILIZADOS	12
	REFERÊNCIAS.....	14

1 INTRODUÇÃO

O gerenciamento de sistemas de informação na área da saúde lida com dados críticos, cujo acesso frequentemente demanda tempo real. Nesse contexto, o Prontuário Eletrônico do Paciente (PEP) atua como um repositório digital que centraliza todo o histórico clínico, diagnósticos e exames de um indivíduo. Essa tecnologia consolida-se como uma ferramenta central na prática clínica, fundamentando a tomada de decisão ágil em momentos emergenciais. Dessa forma, sistemas de PEP exigem alta disponibilidade, baixa latência e rigorosas diretrizes de segurança no manuseio dos dados clínicos.

A conformidade com legislações específicas, como a Lei Geral de Proteção de Dados (LGPD), impõe controles rigorosos sobre as operações de tratamento de informações pessoais, visando assegurar a privacidade do paciente (Lorenzon, 2020). A garantia dessa segurança, contudo, adiciona uma carga de processamento extra à aplicação. A infraestrutura hospitalar deve ser resiliente para que esse custo computacional, somado ao elevado volume de requisições, não eleve a latência a níveis críticos e se converta em um gargalo operacional.

Em situações de emergência ou picos de demanda, arquiteturas monolíticas tradicionais frequentemente apresentam severa degradação de desempenho. Nesse modelo arquitetural, todos os processos operam de forma centralizada e competem simultaneamente pelos mesmos recursos. Consequentemente, o sistema não permite escalar apenas o serviço sobrecarregado, exigindo a replicação da aplicação inteira e resultando em um consumo ineficiente de *hardware* (Newman, 2020). É justamente na garantia da resiliência e na prevenção de falhas sistêmicas que a escolha da arquitetura de *software* se mostra decisiva no ambiente hospitalar (Katkuri, 2025).

Diante dessa problemática, o objetivo central desta pesquisa consiste em realizar um *benchmarking* de persistência poliglota para mensurar, de forma experimental, a eficiência na recuperação de prontuários eletrônicos. Para isso, os experimentos submeterão uma arquitetura monolítica e um ecossistema de microsserviços a variados níveis de carga, simulando desde operações rotineiras até picos críticos. A avaliação comparativa fundamenta-se na extração de métricas

de desempenho, compreendendo a latência de resposta, a vazão de requisições (*throughput*), a taxa de falhas e o consumo de recursos computacionais.

Para atingir essas metas, o trabalho prevê o desenvolvimento de uma interface gráfica de usuário (*front-end*) construída em Angular, focada nas telas de triagem e consulta, além de dois protótipos funcionais de *back-end*: um monolítico e um baseado em microsserviços. Ambos os protótipos de *back-end* serão construídos com o *framework* NestJS e rodarão de forma local em um ambiente computacional orquestrado via contêineres Docker, sem a necessidade de implantação em nuvem. Os dois protótipos utilizarão a mesma estratégia de persistência poliglota, combinando instâncias locais de dois bancos de dados: o PostgreSQL para os dados estruturados de triagem, garantindo consistência transacional, e o MongoDB para os dados não estruturados, como histórico clínico, laudos e metadados da LGPD embutidos. Para não adicionar complexidade desnecessária e isolar as métricas nos testes de estresse, a aplicação não contemplará o desenvolvimento de um sistema de autenticação. A validação dos resultados de desempenho se apoiará em dois métodos estatísticos complementares: o Teste t de Student e a Análise de Variância (ANOVA).

O Teste t de Student será utilizado para verificar se a diferença nas médias de duas amostras independentes é estatisticamente significativa ou apenas fruto de variações aleatórias do ambiente. No contexto desta pesquisa, ele permite comparar diretamente o protótipo monolítico e o ecossistema de microsserviços sob um cenário de carga fixo no Grafana k6. Essa abordagem serve para isolar o impacto puro da escolha arquitetural sobre métricas específicas, como a latência de resposta e a vazão de requisições.

Já a ANOVA de dois fatores será aplicada para analisar o ambiente multivariado dos experimentos. Como os testes simularão desde o uso rotineiro até picos críticos de concorrência, a ANOVA permite avaliar o impacto simultâneo e a interação de duas variáveis independentes: o tipo de arquitetura e a intensidade da carga. Caso essa análise global aponte variações significativas, o teste *post-hoc* de Tukey HSD será acionado logo em seguida. Esse desdobramento é fundamental para comparar os cenários par a par e identificar com precisão a partir

de qual nível de tráfego a degradação de desempenho do sistema passa a ser estatisticamente relevante.

A metodologia selecionada para conduzir a pesquisa é de natureza empírica e aplicada. O comportamento dos protótipos será avaliado sob estresse por meio da ferramenta Grafana k6, responsável por simular e controlar a concorrência de usuários. De forma simultânea, o sistema Prometheus atuará na coleta contínua de métricas de consumo de *hardware*, seguindo os parâmetros práticos da literatura para a extração confiável de métricas em arquiteturas distribuídas (Amaradasa, 2022).

A justificativa para a realização deste estudo ancora-se na necessidade de prover arquitetos de *software* e gestores com evidências empíricas confiáveis sobre resiliência sistêmica. A proposição de ecossistemas descentralizados capazes de sustentar operações médicas ininterruptas alinha-se aos Objetivos de Desenvolvimento Sustentável da ONU, especificamente os voltados à Saúde e Bem-estar (ODS 3) e à Indústria, Inovação e Infraestrutura (ODS 9).

2 TRABALHOS CORRELATOS

A revisão da literatura considerou estudos empíricos sobre a transição para microsserviços, avaliação de desempenho e estruturação de dados na saúde digital, visando posicionar este projeto face aos avanços e lacunas atuais.

Kazanavičius, Mažeika e Kalibatienė (2022) validaram uma abordagem prática para a migração de um banco de dados monolítico legado para uma persistência poliglota distribuída. Os resultados do estudo comprovaram ganhos em escalabilidade e disponibilidade ao alocar diferentes tipos de dados em bases específicas. No entanto, a investigação limitou-se a avaliar atributos de qualidade num cenário corporativo tradicional de *Mainframes*. Este trabalho avança a partir dessa base ao implementar a persistência poliglota (PostgreSQL e MongoDB) especificamente no âmbito clínico, submetendo-a a testes de estresse para avaliar a latência na recuperação de prontuários eletrônicos em cenários emergenciais simulados.

Aderaldo e Mendonça (2022) desenvolveram o ResilienceBench, um ambiente para medir o impacto no desempenho ao aplicar padrões de resiliência em microsserviços. Os autores comprovaram que a injeção de carga é fundamental para revelar gargalos sistêmicos não aparentes em uso normal. Enquanto esse estudo focou no impacto de bibliotecas de código isoladas, como *Retry* e *Circuit Breaker*, o presente projeto apropria-se da metodologia de medição empírica sob estresse, mas a aplica para comparar os estilos arquiteturais, contrastando o desempenho bruto de uma aplicação monolítica em NestJS contra um ecossistema de microsserviços orquestrado via Docker e estressado pelo Grafana k6.

Silva (2022) propõe uma arquitetura de *software* adaptativa voltada para Registros Eletrônicos de Saúde, fornecendo diretrizes relevantes sobre a manipulação e a segurança de dados clínicos. Entretanto, o estudo adota uma abordagem primordialmente conceitual e ilustrativa, deixando uma lacuna crítica: a ausência de validação empírica que demonstre como essa topologia se comportaria operacionalmente sob alta concorrência em um ambiente de pronto-socorro.

Dessa forma, este projeto justifica a sua relevância ao unir as contribuições isoladas desses trabalhos em um experimento prático inédito. A pesquisa preenche as lacunas existentes ao cruzar a estruturação de prontuários médicos (Silva, 2022) e

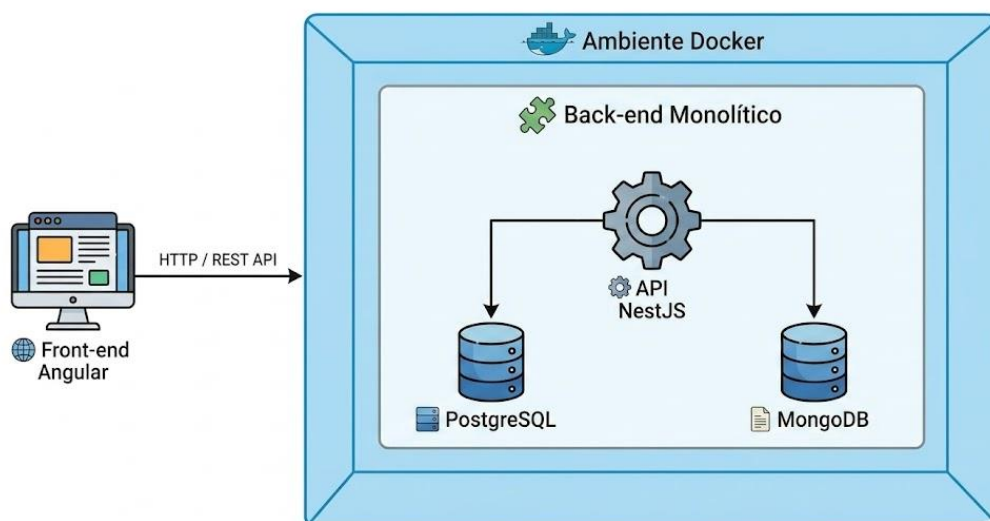
a adoção da persistência poliglota (Kazanavičius; Mažeika; Kalibatienė, 2022) com uma avaliação empírica rigorosa sob sobrecarga (Aderaldo; Mendonça, 2022) mensurando o comportamento e a degradação de desempenho entre a arquitetura monolítica e os microsserviços sob diferentes níveis de carga.

3 O QUE SERÁ FEITO

O escopo deste projeto consistirá na modelagem, construção e avaliação empírica de infraestruturas de *software* para a saúde digital. O trabalho abrangerá a criação de uma interface gráfica *web* desenvolvida no *framework* Angular, focada exclusivamente nos fluxos de triagem e consulta médica. Em conjunto a isso, ocorrerá o desenvolvimento de dois protótipos funcionais de processamento programados em NestJS: o refinamento de uma Arquitetura Monolítica existente e um ecossistema de Microsserviços construído a partir do Método de Estrangulamento (*Strangler Fig Pattern*).

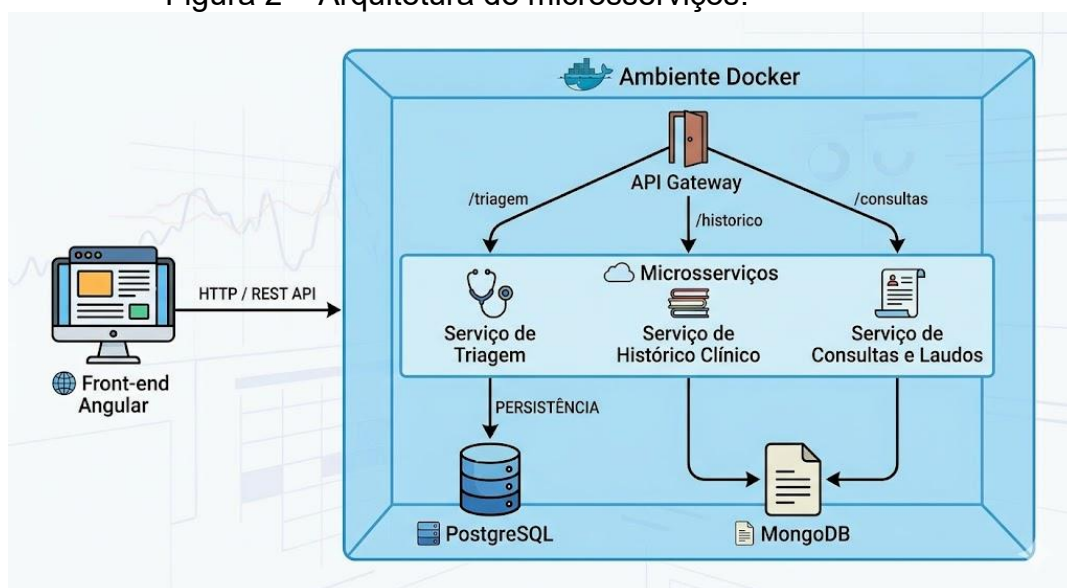
As Figuras 1 e 2 apresentam, respectivamente, a arquitetura monolítica e a arquitetura de microsserviços. Na Figura 1, a interface (*front-end*) conecta-se a um único servidor (*back-end*) que acessa PostgreSQL e MongoDB. Na Figura 2, um API Gateway roteia as requisições para microsserviços independentes (Triagem, Histórico, Laudos), que consultam os bancos conforme a necessidade.

Figura 1 – Arquitetura monolítica.



Fonte: elaborada pelos autores, 2026.

Figura 2 – Arquitetura de microsserviços.



Fonte: elaborada pelos autores, 2026.

Para otimizar os recursos computacionais e viabilizar a persistência poliglota, os sistemas utilizarão instâncias locais do banco relacional PostgreSQL para os dados estruturados e do banco orientado a documentos MongoDB para dados não estruturados, abrangendo laudos e históricos. Para garantir o isolamento e a reprodutibilidade dos experimentos, todo o ecossistema será encapsulado e executado localmente por meio de contêineres Docker, dispensando provedores de nuvem. Os contêineres terão limites de recursos definidos via Docker Compose: cada microsserviço terá alocação máxima de 1 CPU core e 512 MB de RAM; o monolito terá 2 CPU cores e 1 GB de RAM; os bancos de dados (PostgreSQL e MongoDB) terão 1 CPU core e 1 GB de RAM cada. Com o ambiente implantado na máquina local, o projeto submeterá as arquiteturas a testes de estresse com o Grafana k6, enquanto o Prometheus realizará a coleta de métricas e o Grafana exibirá *dashboards* interativos, permitindo uma validação comparativa baseada na Análise de Variância (ANOVA) e no Teste t de Student. Esse ambiente, controlado por limites de recursos e orquestração via Docker Compose, assegura que as condições de execução sejam padronizadas e que os resultados possam ser reproduzidos em diferentes máquinas.

4 O QUE NÃO SERÁ FEITO

O escopo deste projeto não contemplará a implementação de sistemas de autenticação, autorização ou controle de permissões de usuário, garantindo que os testes de carga meçam exclusivamente o desempenho bruto de processamento das arquiteturas e dos bancos de dados. Adicionalmente, não será realizado o *deploy* da aplicação em plataformas de computação em nuvem (como AWS, Azure, Supabase ou MongoDB Atlas), restringindo-se a execução e a avaliação a um ambiente estritamente local orquestrado por contêineres Docker. A arquitetura não utilizará sistemas auxiliares de persistência ou mensageria além do PostgreSQL e MongoDB, nem fará uso de bancos de dados em memória para cache (Redis). Nos testes, não haverá injeção de falhas forçadas (como simulação de quedas de rede, latência artificial ou perda de pacotes), e a orquestração dispensará ferramentas de clusterização avançada para múltiplos nós, como o Kubernetes. Também não será implementada uma camada de segurança adicional (como TLS/HTTPS ou criptografia de dados em trânsito), nem serão utilizados sistemas de *logging* centralizado ou *tracing* distribuído (a exemplo de ELK ou Jaeger). Os experimentos empregarão apenas dados sintéticos gerados pelos próprios *scripts* de carga, sem acesso a informações reais de pacientes, e as instâncias do PostgreSQL e MongoDB serão mantidas com configurações padrão, sem otimizações avançadas de índices ou *tuning* específico, a fim de assegurar a comparabilidade entre as arquiteturas.

5 BENEFÍCIOS

O principal benefício deste projeto é o fornecimento de evidências empíricas concretas que auxiliarão arquitetos de *software* e gestores hospitalares a tomarem decisões embasadas ao escolherem entre as arquiteturas Monolítica e de Microsserviços para cenários médicos de alta criticidade. Essas evidências serão constituídas por métricas experimentais obtidas durante os testes de estresse, abrangendo o tempo de resposta (latência), a vazão de requisições (*throughput*) e o consumo de recursos computacionais, incluindo processamento e memória. Os dados coletados serão submetidos à validação estatística por meio da Análise de Variância (ANOVA) e do Teste t de Student, permitindo verificar se as diferenças observadas entre as arquiteturas apresentam significância estatística e refletem efetivamente o impacto da abordagem arquitetural adotada. Ao isolar a execução em um ambiente local containerizado, o projeto permite uma avaliação mais precisa do desempenho, reduzindo oscilações associadas a redes externas e serviços de nuvem.

Adicionalmente, o estudo demonstra como a aplicação da persistência poliglota otimiza o acesso aos Prontuários Eletrônicos do Paciente (PEP). Essa abordagem fundamenta-se na premissa de que nenhuma tecnologia de banco de dados isolada consegue satisfazer perfeitamente todos os requisitos de capacidade e de alta disponibilidade exigidos por uma infraestrutura médica (Khine; Wang, 2019; Santos; Alencar, 2020). Dessa forma, a combinação estratégica de bancos de dados relacionais e orientados a documentos mitiga gargalos de leitura e escrita, garantindo que informações vitais, abrangendo desde a triagem estruturada até laudos extensos, estejam disponíveis de forma rápida e confiável em momentos de emergência.

Por fim, a pesquisa gera alto valor social por estar em alinhamento direto aos Objetivos de Desenvolvimento Sustentável (ODS) da Organização das Nações Unidas (ONU). O projeto atende ao ODS 3 (Saúde e Bem-estar), ao buscar soluções tecnológicas que sustentem operações clínicas de forma ágil e ininterrupta, e ao ODS 9 (Indústria, Inovação e Infraestrutura), fomentando o desenvolvimento de infraestruturas digitais resilientes e eficientes para o setor da saúde.

6 METAS PARA O TCC 2

O segundo ciclo deste Trabalho de Conclusão de Curso representa a fase de consolidação e experimentação da pesquisa, em que o embasamento teórico construído anteriormente dá lugar à implementação prática e à coleta de evidências empíricas. Nesta etapa, o foco se volta para a evolução de uma arquitetura monolítica em direção a um ecossistema de microsserviços, permitindo investigar, de forma controlada e mensurável, os impactos dessa transição arquitetural no desempenho e na escalabilidade em sistemas de informação em saúde. No presente momento, os protótipos já contam com implementação parcial: o monolítico possui suas rotas essenciais de triagem e consulta funcionando, a migração para microsserviços foi iniciada por meio do padrão de Estrangulamento (*Strangler Fig Pattern*), e o ambiente Docker Compose com limites de recursos já está operacional, assim como as ferramentas k6, Prometheus e Grafana. Para a conclusão do trabalho, foram definidas as seguintes metas:

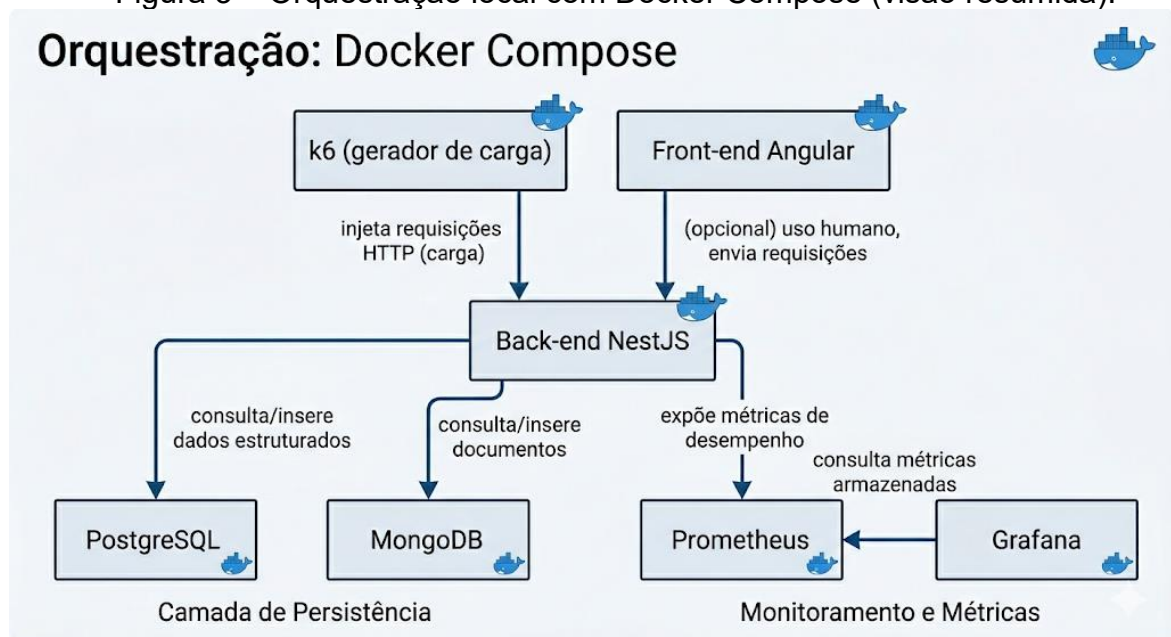
- Refinar o protótipo monolítico de *back-end* existente utilizando o *framework* NestJS;
- Finalizar a aplicação do Método de Estrangulamento (*Strangler Fig Pattern*), concluindo a extração dos módulos e a consolidação do ecossistema de microsserviços;
- Desenvolver a interface gráfica *web (front-end)* em Angular com as telas de triagem e consulta de prontuários;
- Configurar o ambiente de monitoramento com Prometheus e Grafana, além de desenvolver os comandos de injeção de carga massiva na ferramenta Grafana k6;
- Executar as baterias de testes de estresse, coletando dados de latência, vazão de requisições e consumo de *hardware*;
- Analisar estatisticamente os resultados coletados, finalizar a escrita da monografia e apresentar a defesa para a banca examinadora.

7 RECURSOS UTILIZADOS

Os recursos e ferramentas adotados no projeto foram definidos de modo a viabilizar a arquitetura proposta e a execução dos testes, garantindo o isolamento do ambiente experimental, a reprodutibilidade dos resultados e a cobertura de todo o ciclo de desenvolvimento, desde a codificação até a visualização das métricas. As escolhas realizadas visam garantir aderência direta aos requisitos técnicos, evitando dependências desnecessárias que possam comprometer a clareza da avaliação comparativa entre as arquiteturas:

- Linguagens e *Frameworks*: TypeScript, Node.js, NestJS para o processamento de *back-end* e o Angular para a interface *front-end*;
- Bancos de Dados: PostgreSQL para o modelo relacional e MongoDB para o modelo orientado a documentos;
- Infraestrutura e Orquestração: Docker e Docker Compose para execução totalmente local e isolada dos processos, garantindo a ausência de implantação em nuvem (conforme diagrama na Figura 3):

Figura 3 – Orquestração local com Docker Compose (visão resumida).



Fonte: elaborada pelos autores, 2026.

- Testes e Monitoramento: Grafana k6 para injeção de carga e estresse, Prometheus para coleta de métricas e Grafana para visualização de dados via painéis interativos;
- Gestão e Organização: Trello para o acompanhamento das atividades, organização visual do fluxo de trabalho e gerenciamento ágil das entregas entre os integrantes da equipe;
- Ferramentas de Desenvolvimento: Visual Studio Code como ambiente de desenvolvimento integrado para a escrita do código, Git para o controle de versão, GitHub para a hospedagem e colaboração do repositório, e aplicativos como Postman para a simulação e testagem manual das requisições de servidores;
- Equipamentos de *Hardware*: Computadores pessoais dos integrantes do grupo providos de capacidade de processamento e memória suficientes para suportar a execução simultânea dos contêineres e a injeção de carga massiva durante os ensaios locais.

REFERÊNCIAS

ADERALDO, Carlos M.; MENDONÇA, Nabor C. ResilienceBench: um ambiente para avaliação experimental de padrões de resiliência para microsserviços. In: **SIMPÓSIO BRASILEIRO DE REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS (SBRC)**, 40., 2022, Fortaleza, CE. Anais do Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos. Porto Alegre: Sociedade Brasileira de Computação, 2022. p. 65 a 72.

AGÊNCIA NACIONAL DE SAÚDE SUPLEMENTAR. Resolução Normativa nº 572, de 23 de fevereiro de 2023. Dispõe sobre o Prontuário Eletrônico do Paciente (PEP) e o Registro Eletrônico de Saúde (RES). Diário Oficial da União, Brasília, DF, 2023. AMARADASA, G. W. G. S. V. Reliable performance metrics for microservice-based applications. 2022. Dissertação (Mestrado) – University of Moratuwa, Moratuwa, 2022. 13 p

KATKURI, Rameshreddy. Revolutionizing healthcare with secure and scalable microservices. **World Journal of Advanced Research and Reviews**, v. 26, n. 1, p. 1731–1739, 2025.

KAZANAVIČIUS, J.; MAŽEIKA, D.; KALIBATIENĖ, D. An Approach to Migrate a Monolith Database into Multi-Model Polyglot Persistence Based on Microservice Architecture: A Case study for Mainframe Database. **Applied Sciences**, [S.l.], v. 12, n. 12, p. 6189, 2022. DOI: 10.3390/app12126189. Disponível em: <https://doi.org/10.3390/app12126189>. Acesso em: 15 abr. 2026.

KHINE, Pwint Phyu; WANG, Zhaoshun. A review of polyglot persistence in the big data world. **Information**, [S.l.], v. 10, n. 4, p. 141, 2019. DOI: 10.3390/info10040141. Disponível em: <https://doi.org/10.3390/info10040141>. Acesso em: 29 abr. 2026.

LORENZON, Laila Neves. Análise comparada entre regulamentações de dados pessoais no Brasil e na União Europeia (LGPD e GDPR) e seus respectivos instrumentos de enforcement. 2020. Dissertação (Mestrado em Direito) – Fundação Getulio Vargas, Rio de Janeiro, 2020. 14 p.

NEWMAN, Sam. **Migrando sistemas monolíticos para microsserviços**: padrões evolutivos para transformar seu monólito. São Paulo: Novatec Editora, 2020. 288 p.

SANTOS, Keven Leone dos; ALENCAR, Andrêza Leite de. Aplicabilidade da persistência poliglota em sistemas potencialmente escaláveis. **Revista de Engenharia e Pesquisa Aplicada**, Recife, v. 5, n. 1, p. 78 a 87, 2020.

SILVA, Marcio Alexandre Pereira da. Uma Arquitetura de Software Adaptativa baseada em Arquétipo OpenEHR para Sistemas de Informação em Saúde. 2022. Tese (Doutorado em Ciência da Computação). Centro de Informática, Universidade Federal de Pernambuco, Recife, 2022. Disponível em: <https://repositorio.ufpe.br/handle/123456789/48275>. Acesso em: 30 abr. 2026.